

Diagnóstico técnico: error "There is no active transaction"

Por qué el problema está en la página de Streaming (revendedores), no en streaming.reborxstore.com (pryorreos)

Sistema bajo análisis: Página de Streaming / Revendedores (proyecto externo, no pryorreos) Fecha: 8 de agosto de 2026

Conclusión: Los errores "There is no active transaction" (panel de inventario) y "No se pudo completar la compra. Intenta de nuevo." (página de compra) **no los genera pryorreos**. Son errores de manejo de transacciones de base de datos que ocurren dentro del código de la página de Streaming, casi con certeza en la función que procesa la compra/asignación, justo después de haber integrado la llamada al bot de códigos. Pryorreos fue verificado en producción durante este mismo diagnóstico: responde rápido y siempre con JSON válido.

1. Los dos errores reportados

Dónde aparece	Mensaje	Idioma / origen probable
Panel "Cuentas de streaming" (inventario, vista admin)	⚠ There is no active transaction	Inglés — texto literal de una excepción PDOException de PHP, no traducido. No es un mensaje que la aplicación redacta a propósito.
Página "Comprar del stock de la tienda" (compra, vista del revendedor)	⚠ No se pudo completar la compra. Intenta de nuevo.	Mensaje genérico en español, probablemente el catch general que envuelve a la compra cuando algo dentro falla — incluida la excepción del punto anterior.

El propio cliente confirmó que **antes de conectar el bot de códigos estos errores no aparecían**. Esto ubica el momento exacto en que se introdujo el problema: al agregar la llamada HTTP al endpoint de pryorreos dentro del flujo de compra/asignación.

2. Evidencia de que pryorreos no es la causa

2.1 El mensaje de error no puede salir de pryorreos

El endpoint `api/external/asignaciones.php` envuelve toda su lógica en un único bloque `try/catch (Throwable)` y **siempre** responde con JSON controlado, nunca con el texto crudo de una excepción de PHP/PDO:

```
} catch (Throwable $exception) {
    http_response_code(500);
    echo json_encode(['success' => false, 'message' => 'No fue posible procesar la solicitud.'],
JSON_UNESCAPED_UNICODE);
}
```

Es técnicamente imposible que este endpoint devuelva el texto "There is no active transaction" — ese es un mensaje que solo puede originarse en un PDOException lanzado por **otro** código PHP, al llamar `$pdo->commit()` o `$pdo->rollback()` cuando no hay una transacción abierta en esa conexión.

2.2 Prueba de latencia y disponibilidad en vivo

Se hicieron 3 llamadas directas al endpoint en producción durante este diagnóstico:

```
HTTP:401 total:0.520109s
HTTP:401 total:0.367075s
HTTP:401 total:0.377965s
```

Tiempo de respuesta entre 0.37 y 0.52 segundos, siempre con JSON válido. Esto descarta que el endpoint esté colgado, lento, o devolviendo respuestas corruptas que puedan estar rompiendo el flujo del otro sistema.

2.3 Evidencia de que el flujo completo ya funcionó correctamente

En una prueba real anterior a este error, se confirmó en la base de datos de pryorreos que una venta activó la asignación automática correctamente:

Evento	Marca de tiempo
Último uso registrado del token de API	2026-08-08 02:47:33
Asignación creada automáticamente en pryorreos	2026-08-08 02:47:33

Ambos eventos coinciden al segundo, confirmando que el endpoint recibió la llamada, la procesó y devolvió una respuesta exitosa sin inconvenientes en esa ejecución.

3. Qué significa técnicamente "There is no active transaction"

Es el mensaje exacto que lanza PDO (la extensión de base de datos de PHP) cuando el código ejecuta:

```
$pdo->commit();
// o
$pdo->rollBack();
```

en un momento en que `$pdo->inTransaction()` es `false` — es decir, no hay ninguna transacción abierta en ese momento sobre esa conexión. Las causas más comunes son:

- Se llama a `commit()` o `rollBack()` **dos veces** para la misma operación (por ejemplo, una vez en el flujo normal y otra vez dentro de un bloque `catch` o `finally`).
- Una excepción ocurre **después** de que la transacción ya fue confirmada (`commit()`), pero el código de manejo de errores intenta hacer `rollBack()` igual, asumiendo que todavía estaba abierta.
- Se abre una **segunda transacción anidada** sobre la misma conexión sin usar `SAVEPOINT` — MySQL/PDO no soportan transacciones anidadas de forma nativa, así que la segunda apertura puede dejar el estado inconsistente.
- La conexión a la base de datos se cierra o se reinicia (por ejemplo, por un timeout) mientras la transacción estaba abierta, y luego el código intenta cerrarla igual.

4. Diagnóstico más probable

Hipótesis principal: el código que procesa la compra/asignación en la página de Streaming abre una transacción de base de datos, y **dentro de esa misma transacción** se agregó la llamada HTTP saliente al bot de códigos (a `api/external/asignaciones.php`). Si esa llamada HTTP tarda, lanza una excepción, o el manejo de la respuesta no está bien controlado, el bloque `catch / finally` de ese flujo termina llamando a `commit()` o `rollBack()` en un momento en que la transacción ya no está activa (por ejemplo, porque ya se había cerrado antes, o porque nunca llegó a abrirse debido a un error previo en el mismo request).

Esto explica perfectamente por qué el error apareció **justo después** de conectar el bot y no antes: el código nuevo que dispara la llamada HTTP fue insertado en un punto del flujo transaccional que no estaba preparado para manejar una operación externa (llamada de red) en el medio.

5. Pasos técnicos para que el otro programador lo resuelva

- ☐ **Ubicar el bloque de transacción.** Buscar en el código de "procesar compra" / "asignar cuenta a revendedor" dónde se llama a `beginTransaction()`, `commit()` y `rollback()`, y confirmar en qué línea, dentro de ese bloque, se agregó la llamada HTTP al bot de códigos.
- ☐ **Sacar la llamada al bot de adentro de la transacción.** Reordenar el flujo así:

```
$pdo->beginTransaction();
// descontar inventario, crear registro de venta, etc.
$pdo->commit();

// FUERA de la transacción, después del commit:
try {
    llamarBotDeCódigos($accountEmail, $buyerEmail, 'asignar');
} catch (Throwable $e) {
    // registrar el error en un log aparte
    // NO afecta la venta, que ya quedó confirmada
}
```

Esto es una buena práctica general: las llamadas de red (HTTP) nunca deberían ejecutarse dentro de una transacción de base de datos, porque su duración es impredecible y un fallo de red no debería poder revertir una operación que ya es válida en la base de datos.

- ☐ **Revisar los bloques `catch` / `finally` existentes.** Verificar que ningún `catch` llame a `rollback()` o `commit()` sin antes comprobar:

```
if ($pdo->inTransaction()) {
    $pdo->rollback();
}
```

- ☐ **Ponerle timeout a la llamada HTTP saliente.** Si usan cURL, agregar `CURLOPT_TIMEOUT` (por ejemplo, 10 segundos) y `CURLOPT_CONNECTTIMEOUT`, para que una llamada lenta no cuelgue el request completo. Ya se confirmó que el endpoint responde en menos de 1 segundo, así que un timeout de 10s da margen de sobra sin bloquear al usuario.
- ☐ **Confirmar que un fallo del bot no bloquee la compra.** Si el bot devuelve un error (por ejemplo, 404 porque el comprador todavía no está vinculado como vendedor en prycorreos), la compra debería completarse igual del lado de la tienda, y el fallo de asignación quedar solo registrado en un log para revisión manual — no debería mostrarle al comprador "No se pudo completar la compra".

6. Cómo confirmar que quedó resuelto

1. Hacer una compra de prueba en la página de Streaming con un comprador ya vinculado como vendedor en prycorreos.
2. Verificar que ya no aparece ningún mensaje de error ni en el panel de inventario ni en la página de compra.

3. Confirmar en prycorreos (pestaña "Usuarios Registrados" del comprador, o directamente con nosotros) que la cuenta quedó asignada correctamente.

Diagnóstico realizado sobre el código real de prycorreos (`api/external/asignaciones.php`) y verificación en vivo del endpoint en producción. La causa raíz descrita en la sección 4 y los pasos de la sección 5 corresponden al código de la página de Streaming, al cual este documento no tuvo acceso directo — se recomienda que el programador de ese sistema confirme la ubicación exacta del bloque de transacción antes de aplicar el cambio.